# Find on Naïve

Setup and operating guide for the Find step (property-tax desk) · taxappeal-find · November 2026

## 1. What runs where

Find is one agent in your Naïve organization. A schedule wakes it every 6 hours; it runs the `taxfind` program on its own Linux computer, which reads Florida's county tax rolls, flags over-assessed homes, and posts each new case as a card on the Qualify agent's board. Find never contacts owners and never files anything.

| Piece | Lives in | Does |
|---|---|---|
| Find agent | Naïve (its own computer) | runs `taxfind`; posts cards; reports each run |
| Schedule (deployment) | Naïve | wakes Find at :17 past every 6th hour, New York time |
| Qualify board | Smaran's Qualify agent | receives one card per case; the card key is the case id, so nothing posts twice |
| County rolls + sales | Florida DOR data portal (free) | the data Find screens; new files are picked up automatically |
| Home-value check (optional) | Naïve data catalogue, then Apify Zillow, then Redfin | second opinion on the best candidates, within a dollar cap |

## 2. Before you start

| You need | How to get it |
|---|---|
| Node + npm, Python 3.10+, jq | Mac: `brew install node python jq` |
| A Naïve API key | your Naïve dashboard (app.usenaive.ai) |
| Your organization on a plan | `naive plan show`; without a plan every command is refused |
| The Qualify agent's id (agt_…) | from Smaran; Find posts its cards to that agent's board |
| Optional: an Apify token | Apify console → Settings → API & Integrations; turns on the Zillow/Redfin check |

## 3. Setup in five steps

### 1. Install (once)

```
npm i -g @usenaive-sdk/cli
cd ~/Desktop/taxappeal-find && pip install -e .
```

### 2. Log in

```
printf '%s' "YOUR_NAIVE_KEY" | naive login
naive whoami
naive plan show
```

### 3. Create the Find agent, its secrets and its schedule (one command, from the project folder)

```
QUALIFY_AGENT=agt_xxx COUNTIES=Lee INCLUDE_CLOSED=1 ./naive/setup.sh
```

| Option | Meaning |
|---|---|
| `QUALIFY_AGENT` | required: the Qualify agent's id |
| `COUNTIES=Lee` | demo: run only these counties. Leave it out for all 67 (the first full pass downloads about 1 GB and takes about 25 minutes) |
| `INCLUDE_CLOSED=1` | demo: post cards even though the 2026 filing window has closed. Without it, 2026 cases are held and nothing is posted |
| `APIFY_TOKEN=…` | optional: stores your Apify token so Find can ask Zillow/Redfin |
| `FIND_NAIVE_KEY=…` | optional: a key the program uses to post cards itself; without it the agent posts them with its board tool |
| `TAXFIND_GIT=…` | optional: install the code from your team's git repo instead of an uploaded zip |

The script prints the Find agent id and a deployment id. Keep both.

### 4. Run a pass now instead of waiting for the schedule

```
naive deploy run <deployment id>
```

### 5. Watch it and check the cost

```
naive session list --agent <find agent id>
naive agent spend <find agent id>
```

## 4. What setup.sh creates

| Step | Creates | Details |
|---|---|---|
| 1/5 | skill `taxfind-find` | from `naive/SKILL.md`: teaches the agent the commands, tiers and signals |
| 2/5 | file `taxappeal-find.zip` | the code only; data, run outputs and .env are excluded (skipped when TAXFIND_GIT is set) |
| 3/5 | agent `find` | from `naive/find.agent.yaml`; re-running updates it by name |
| 4/5 | vault `find-secrets` | APIFY_TOKEN (for api.apify.com) and NAIVE_API_KEY (for api.vetta.sh), each used only for its host |
| 5/5 | deployment | the schedule: every 6 hours, $3 cap per run, low-priority ("loose") window |

## 5. The agent

| Setting | Value | Why |
|---|---|---|
| Model | `openai/gpt-oss-120b` | the work is a program run with bash; a mid-size model is enough (`naive models list` shows options) |
| Budget | $30 per month, $3 per run | hard ceiling on model and data spend; the program also caps lookups |
| Window | loose | scheduled background work; nobody is waiting |
| Default tool permission | deny | closed by default; only the tools below are open |

| Tool | Permission | Used for |
|---|---|---|
| `bash, read, write, ls, find` | allow | install and run `taxfind`, read its outputs |
| `fetch_file` | allow | pull the uploaded code zip |

| Tool | Permission | Used for |
|---|---|---|
| `publish_file` | allow | publish each run's candidates.csv and summary.json |
| `board_read, board_write` | allow | post cases to Qualify; flag blocked counties on its own board |
| `data_search, data_describe` | allow | find Naïve's home-value data tool |
| `data_run` | ask | paid per call, so a person approves |
| `ask_operator` | allow | ask a person before anything irreversible or over budget |
| `session_spend` | allow | report what the run cost |
| `web_fetch` | allow, floridarevenue.com only | read the state portal if needed |

## 6. What the agent does each run

| # | Step |
|---|---|
| 1 | Install the code if missing: from `TAXFIND_GIT`, else the uploaded zip. Data lives in `$HOME/find-data`, so the cache and database survive between runs. |
| 2 | Run `taxfind watch --once --handoff board`: only counties that posted new files are processed; new strong/possible cases are posted to the Qualify board. (With COUNTIES set, it runs `taxfind run --counties …` instead.) |
| 3 | Publish the run's `candidates.csv` and `summary.json`. |
| 4 | Report in five lines or fewer: counties, homes screened, strong/possible counts, yearly savings, cards posted, money spent. |

### Rules it follows

| Situation | What it does |
|---|---|
| A county fails | retries once; still failing, it puts a blocked card on its own board and carries on |
| Strong cases in a county jump more than 3x since the last run | posts a blocked card asking a person to look before Qualify acts |
| Tiers review, new_build, cap_absorbs | never posted to Qualify; kept in the database for people |
| No NAIVE_API_KEY on its computer | runs with --handoff file and posts up to 50 cards itself with board_write (key = case id) |
| Owner contact data | public owner of record and tax-bill mailing address only; no phone numbers, emails or social profiles |
| Anything irreversible or over budget | asks a person first (ask_operator) |

## 7. What Qualify receives

One card per case. Title: state, county, year, address, estimated yearly savings and tier, e.g. `FL Lee 2026 · 5404 SW 28TH PL, CAPE CORAL · ~$6,868/yr · strong`. Body: a short summary, then the full case as JSON (the schema is in `docs/HANDOFF.md`; print it with `taxfind schema`).

| Held back (not posted) | Why |
|---|---|
| Counties whose self-check is "weak" | flagged homes there did not sell below county value more often than others, so the screen isn't trustworthy there yet |
| Cases whose filing window has closed | nothing can be filed; override with INCLUDE_CLOSED=1 / `--include-closed` for demos |
| Anything already posted | the card key is the case id |

## 8. Settings

Set these in the deployment prompt (setup.sh does it), in a `.env` file in the project folder, or in your shell.

| Variable | Default | Meaning |
|---|---|---|
| `NAIVE_API_KEY` | — | lets the program post cards and run data tools |
| `NAIVE_QUALIFY_BOARD` | — | Qualify agent id (agt_…) or board id (brd_…) |
| `NAIVE_QUALIFY_AGENT` | — | optional: assign each card to this agent |
| `NAIVE_REAL_ESTATE_TOOL` | — | Naïve home-value tool id, from `taxfind naive-tools` |
| `NAIVE_API_BASE_URL` | `https://api.vetta.sh` | Naïve API address |
| `APIFY_TOKEN` | — | turns on Zillow/Redfin via Apify |
| `FIND_AVM_PROVIDERS` | `naive,apify_zillow,apify_redfin` | home-value sources, in order (add mock for keyless demos) |
| `FIND_AVM_BUDGET_USD` | 5 | dollar cap per run for home-value lookups |
| `FIND_AVM_MAX_LOOKUPS` | 300 | best candidates looked up per county |
| `FIND_HANDOFF_MAX_CARDS` | 200 | cards posted per run |
| `FIND_HANDOFF_INCLUDE_CLOSED` | 0 | 1 = post even after the filing window closed |
| `FIND_DATA_DIR` | `./data` | cache, database and outputs |
| `FIND_DRY_RUN` | 0 | 1 = write Naïve requests to data/naive_outbox.jsonl instead of sending |

## 9. Command cheat sheet

### Naïve

| Command | Does |
|---|---|
| `naive login / whoami / plan show` | sign in, check who you are, check the plan |
| `naive agent apply -f naive/` | create or update the agent after editing `find.agent.yaml` |
| `naive agent show <id> --format yaml` | see the live agent config |
| `naive agent spend <id>` | what the agent has spent |
| `naive deploy run <id>` | run the schedule now |
| `naive deploy pause <id> / resume <id>` | stop or restart the schedule |

| Command | Does |
|---|---|
| `naive deploy runs <id>` | past scheduled runs |
| `naive session list --agent <id>` | the agent's runs (sessions) |
| `naive session get <session id> --human` | one run in detail |
| `naive skill push --slug taxfind-find --file naive/SKILL.md` | update the skill |
| `naive models list` | models your organization can use |

### taxfind (on your laptop or the agent's computer)

| Command | Does |
|---|---|
| `taxfind sources` | counties the state has published |
| `taxfind run --counties Lee [--no-avm] [--handoff board]` | one pass over some or all counties |
| `taxfind watch --once --handoff board` | autonomous pass: only counties with new files |
| `taxfind handoff [--county-code 46] [--include-closed]` | post not-yet-posted cases (the backlog) |
| `taxfind naive-tools [--tool <id>]` | search Naïve's real-estate data tools; show one tool's inputs |
| `taxfind upload leads.csv --county Lee` | match a list or a tax notice to parcels and give a verdict |
| `taxfind status` | recent runs and money spent |
| `taxfind calendar` | which states file in which month |
| `./scripts/demo.sh Lee` | the full cycle end to end, no keys needed |

## 10. Demo day

| Goal | Do this |
|---|---|
| Show it on Naïve | setup with `COUNTIES=Lee INCLUDE_CLOSED=1`, then `naive deploy run <id>`; cards appear on Qualify's board when the run finishes |
| Show it without a key | `./scripts/demo.sh Lee`: real data, mock home values (labelled mock), cards written to data/naive_outbox.jsonl |
| Rehearse the Naïve calls | `FIND_DRY_RUN=1 taxfind run --counties Lee --handoff board --include-closed` |
| Use Naïve's own home values | `taxfind naive-tools` → pick the tool → set `NAIVE_REAL_ESTATE_TOOL` |

## 11. Confirm at the event, and fixes

These could not be tested without a live key:

| Check | If it fails |
|---|---|
| The agent's computer can reach floridarevenue.com and api.apify.com from bash | run `taxfind run` on your laptop instead and post with `taxfind handoff --include-closed` (needs NAIVE_API_KEY locally) |
| Vault secrets appear as environment variables on the agent's computer | nothing to do: the agent falls back to posting cards with board_write; for Apify, set the token in the dashboard |

| Check | If it fails |
|---|---|
| A Naïve real-estate data tool returns a value for an address | remove `naive` from FIND_AVM_PROVIDERS; Apify Zillow/Redfin still work |

| Symptom | Fix |
|---|---|
| Every command is refused | `naive plan show`: the organization needs a plan |
| No cards on Qualify's board | 2026 window closed (use INCLUDE_CLOSED=1), the county is "weak", or the cards were already posted |
| A run stops partway | budget hit: `naive agent spend <id>`; raise cap_usd / max_task_usd in find.agent.yaml, then `naive agent apply -f naive/` |
| First pass is slow | normal: about 1 GB and 25 minutes for all 67 counties; later passes only redo counties with new files |
| You changed the agent's instructions | edit naive/find.system.md, then `naive agent apply -f naive/` |
| Stop it running | `naive deploy pause <deployment id>` |

## Appendix A · naive/find.agent.yaml

```yaml
# The Find agent on Naive. Apply with:  naive agent apply -f naive/
# Field names follow @usenaive-sdk/cli 1.0 (`agent apply` upserts by name).
name: find
description: >
  Find desk. Watches state property-tax rolls, flags over-assessed homes, matches each to its owner
  of record, gets a Zillow/Redfin second opinion on the shortlist, and posts candidates to the
  Qualify agent's board. Never contacts property owners.

# Most of the work is a Python program run with bash, so a mid-size model is plenty.
# `naive models list` shows what your org can use.
model: openai/gpt-oss-120b
window: loose            # scheduled background work; no one is waiting on it

budget:
  cap_usd: 30            # per period, model + data calls
  max_task_usd: 3        # one scheduled run
  period: month

system_file: naive/find.system.md

skills:
  - taxfind-find         # pushed by setup.sh from naive/SKILL.md

tools:
  default_config:
    permission: deny     # closed by default; only what Find needs is opened below
  configs:
    bash:          { enabled: true, permission: allow }
    read:          { enabled: true, permission: allow }
    write:         { enabled: true, permission: allow }
    ls:            { enabled: true, permission: allow }
    find:          { enabled: true, permission: allow }
    fetch_file:    { enabled: true, permission: allow }   # pull the code zip when there is no git repo
    publish_file:  { enabled: true, permission: allow }   # publish each run's candidates.csv + summary
    board_read:    { enabled: true, permission: allow }
    board_write:   { enabled: true, permission: allow }
    data_search:   { enabled: true, permission: allow }
    data_describe: { enabled: true, permission: allow }
    data_run:      { enabled: true, permission: ask }     # paid per call; the program has its own cap too
    ask_operator:  { enabled: true, permission: allow }   # a human decides anything irreversible
    session_spend: { enabled: true, permission: allow }
    web_fetch:
      enabled: true
      permission: allow
      config:
        allowed_domains: [floridarevenue.com]
        max_content_tokens: 4000

metadata:
  desk: property_tax
  step: find
  hands_off_to: qualify
```

# Appendix B · naive/find.system.md (the agent's instructions)

```
You are **Find**, the first agent in a property-tax appeal company. Your one job: find homes the
county has over-assessed, attach the owner of record, and hand each case to the **Qualify** agent.
You never contact owners, never file anything, and never spend outside your budget.

## Each run

1. **Get the code.** In your computer, if `taxfind` is not installed:
   - if `$TAXFIND_GIT` is set: `pip install "git+$TAXFIND_GIT"`
   - otherwise fetch the org file named `taxappeal-find.zip` with `fetch_file`, unzip it and
     `pip install ./taxappeal-find`.
   Keep `FIND_DATA_DIR=$HOME/find-data` so the cache and database survive between runs.

2. **Run one pass.** `taxfind watch --once --handoff board`
   - It checks the state's data portal, processes only counties that posted new files since the
     last pass, screens every house and condo, looks up a home-value estimate for the best
     candidates (within `FIND_AVM_BUDGET_USD`), stores everything, and posts new strong/possible
     cases to the Qualify board (`NAIVE_QUALIFY_BOARD`). Posting is idempotent by case id.
   - Nothing new posted? That is normal outside roll season. Say so in one line and stop.
   - No `NAIVE_API_KEY` in your computer? Run `taxfind watch --once --handoff file` instead, then
     read the run's `candidates.jsonl` and post each new `strong`/`possible` case yourself with
     `board_write` on the Qualify board: title = the case's address + yearly savings, body = the
     JSON line, key = its `case_id` (so a repeat run never posts it twice). At most 50 per run,
     highest `estimate.annual_savings` first.

3. **Publish the run.** `publish_file` the run's `candidates.csv` and `summary.json`
   (paths are printed at the end of the run) with the title `Find run <run_id>`.

4. **Report** in five lines or fewer: counties processed, homes screened, strong / possible
   counts, estimated yearly savings for strong cases, cards posted, money spent
   (`taxfind status` shows spend).

## Rules

- A county that fails: retry it once (`taxfind run --counties <name>`). Still failing: put a card
  on your own board with status `blocked` and the error, then continue with the rest.
- Never change detection thresholds on your own. If strong cases in a county jump by more than
  3x versus its last run, post a `blocked` card asking a person to look before Qualify acts.
- Tiers `review`, `new_build` and `cap_absorbs` are never posted to Qualify; they stay in the
  database for people to inspect.
- Owner data is the public owner of record and tax-bill mailing address only. Do not look up
  phone numbers, emails or social profiles.
- Use `ask_operator` before anything irreversible or any spend above your task budget.
```

# Appendix C · naive/SKILL.md

```
---
name: taxfind-find
description: How to run and read the taxfind Find program (over-assessed homes from public rolls).
---

# taxfind — Find step

## Commands
- `taxfind sources` — counties the state has published, with roll year and stage.
- `taxfind run --counties Lee,Collier [--no-avm] [--handoff file|board]` — one pass.
- `taxfind watch --once --handoff board` — autonomous pass: only counties with new files.
- `taxfind upload <file> --county <name>` — match a list (CSV/XLSX), a TRIM notice (PDF) or pasted
  text to parcels and say which ones have a case.
- `taxfind export --tiers strong,possible --out ./export` — JSONL + CSV from the database.
- `taxfind status` — recent runs and spend. `taxfind naive-tools` — find a home-value data tool.
- `taxfind schema` — the JSON schema of a candidate (what Qualify receives).

## Tiers
- **strong** — both signals (neighbors and sales), clean data, at least $500/yr at stake. Mail-ready.
- **possible** — both signals, less money. **review** — big gap or high value, or the
  Zillow/Redfin estimate disagreed: a person looks first. **cap_absorbs** — over-assessed, but the
  homestead cap means no savings. **new_build** — built in the last two years, set aside.

## Signals in each candidate
- `neighbor_ratio` — county $/sq ft ÷ median of similar homes in the same appraisal neighborhood.
- `sales_ratio` — county value ÷ (comparable sales time-adjusted to Jan 1 × county's normal level).
- `avm_ratio` — county value ÷ (Zillow/Redfin estimate × county's normal level). `avm_check`:
  confirmed (≥1.05), inconclusive, disputed (<0.95, demoted to review), no_unit (condo, unit unknown).

## Environment
`FIND_DATA_DIR`, `NAIVE_API_KEY`, `NAIVE_QUALIFY_BOARD` (the Qualify agent's `agt_` id),
`NAIVE_REAL_ESTATE_TOOL`, `APIFY_TOKEN`, `FIND_AVM_PROVIDERS` (default `naive,apify_zillow,apify_redfin`),
`FIND_AVM_BUDGET_USD` (default 5), `FIND_AVM_MAX_LOOKUPS` (default 300 per county).
```

## Appendix D · naive/setup.sh

```
#!/usr/bin/env bash
# One-time setup of the Find agent on Naive. Run from the repo root after `naive login`.
#   QUALIFY_AGENT=agt_...  ./naive/setup.sh
#   demo:  QUALIFY_AGENT=agt_... COUNTIES=Lee INCLUDE_CLOSED=1 ./naive/setup.sh
# Optional: TAXFIND_GIT=https://github.com/<team>/taxappeal-find  (else the code is uploaded as a zip)
set -euo pipefail
command -v naive >/dev/null || { echo "install the CLI: npm i -g @usenaive-sdk/cli"; exit 1; }
command -v jq >/dev/null || { echo "needs jq (Mac: brew install jq)"; exit 1; }
: "${QUALIFY_AGENT:?set QUALIFY_AGENT to the Qualify agent id (agt_...) - Find posts cards to its board}"

naive whoami >/dev/null
naive plan show >/dev/null || { echo "your org needs a plan: naive plan show / naive plan set"; exit 1; }

echo "1/5 skill"
naive skill push --slug taxfind-find --file naive/SKILL.md --description "Run and read the taxfind Find program"
      >/dev/null

echo "2/5 code"
if [ -z "${TAXFIND_GIT:-}" ]; then
  rm -f /tmp/taxappeal-find.zip
  # code only: no data, no run outputs, no secrets
  (cd .. && zip -qr /tmp/taxappeal-find.zip "$(basename "$OLDPWD")" -x '*/data/*' '*/output/*' '*/.git/*'
      '*/__pycache__/*' \
     '*.egg-info/*' '*/.env' '*/.venv/*' '*/.pytest_cache/*' '*_matched.csv')
  naive file upload --file /tmp/taxappeal-find.zip --name taxappeal-find.zip --title "taxfind code" >/dev/null
fi

echo "3/5 agent"
AGENT_ID=$(naive agent apply -f naive/ | jq -r '.. | objects | select(.object? == "agent") | .id' | head -1)
echo "    find agent: $AGENT_ID"

echo "4/5 secrets (injected for their host only; the value never enters the transcript)"
VAULT_ID=$(naive vault list | jq -r '.data[]? | select(.display_name=="find-secrets") | .id' | head -1)
[ -n "$VAULT_ID" ] || VAULT_ID=$(naive vault create --name find-secrets | jq -r .id)
if [ -n "${APIFY_TOKEN:-}" ]; then
  printf '%s' "$APIFY_TOKEN" | naive vault set "$VAULT_ID" --env APIFY_TOKEN --host api.apify.com >/dev/null \
    || echo "    (could not store APIFY_TOKEN - add it to the find-secrets vault in the dashboard)"
fi
if [ -n "${FIND_NAIVE_KEY:-}" ]; then   # a scoped key the program uses to post cards and run data tools
  printf '%s' "$FIND_NAIVE_KEY" | naive vault set "$VAULT_ID" --env NAIVE_API_KEY --host api.vetta.sh >/dev/null \
    || echo "    (could not store NAIVE_API_KEY - the agent will post cards with board_write instead)"
fi

echo "5/5 schedule: every 6 hours at :17, America/New_York"
# Demo options: COUNTIES=Lee runs just those counties (the first statewide pass downloads ~1 GB and takes
# ~25 min); INCLUDE_CLOSED=1 posts cards even though the 2026 filing window has closed.
PROMPT="Run your Find pass now."
[ -n "${COUNTIES:-}" ] && PROMPT="$PROMPT This time run: taxfind run --counties $COUNTIES --handoff
      board${INCLUDE_CLOSED:+ --include-closed}."
PROMPT="$PROMPT Environment: FIND_DATA_DIR=\$HOME/find-data NAIVE_QUALIFY_BOARD=$QUALIFY_AGENT
      ${INCLUDE_CLOSED:+FIND_HANDOFF_INCLUDE_CLOSED=1 }${TAXFIND_GIT:+TAXFIND_GIT=$TAXFIND_GIT}"
naive deploy create --agent "$AGENT_ID" --cron "17 */6 * * *" --timezone America/New_York \
  --prompt "$PROMPT" --budget-usd 3 --window loose | jq '{id, cron, next_run_at}'

echo "done. Trigger a pass now with: naive deploy run <deployment id>"
```